Using Autoencoders for Dimension Reduction

Dorian SOUSA CALVO
dorian.sousa.001@student.uni.lu
University of Luxembourg
LUX

Abstract

This project's goal was to investigate various autoencoder architectures for dimensionality reduction and clustering of the MNIST dataset. This report presents the implemented AEs, including a stacked autoencoder (SAE) from the DEC paper [10], a convolutional autoencoder and a variational autoencoder (VAE). Evaluation of the project focuses on how well the resulting clusters separated the dataset into its true labels, more specifically I will evaluate with clustering accuracy, NMI and ARI. The convolutional VAE performed best across all metrics achieving 85.73% in clustering accuracy, 72.92% in ARI and 75.53% in NMI.

1 Introduction

Labeling data is expensive, but labels are useful for evaluation. In my project I wanted to explore unsupervised learning and how well clustering would be able to recover the classes and group the datapoints in it. This could allow datasets to be grouped or labeled using unsupervised learning.

For this, I explored autoencoders for their application in dimensionality reduction by mapping features to a latent space. This latent space will then be clustered using the k-means algorithm, with the evaluations focusing on how well the assigned clusters match the ground truth.

2 Related Work

The main project idea was inspired by the paper [10], which used a stacked autoencoder (SAE) to learn a mapping of features into a lower dimension latent space. Their SAE implementation and hyperparameters have been used to implement an initial replication of the model so that it can be used as a baseline and a foundation to experiment on. The base I set was their AE+k-means with 81.70% clustering accuracy.

The Variational Autoencoder (VAE) was introduced in [4] and is an AE whose encoder outputs a distribution and utilizes a loss function that combines reconstruction loss and KL-divergence regularization.

[1] introduced the β hyperparameter to control the KL-divergence in the VAE's ELBO loss function.

[2] demonstrates how AEs can be utilized to learn low-dimensional representations and can outperform PCA.

[5] paper's results show how convolutional layers can be effective for image feature extraction.

MNIST[6] dataset is the only dataset used in this project, it is a commonly used benchmark and allows the project to focus on experimenting with various architectures. For the clustering algorithm I chose k-means[8]. Adjusted rand index (ARI) [3] and normalized mutual information (NMI) [9] were used to compare the predicted clusters against the ground-truth labels.

AdamW[7] was used as an optimizer on models that do not build on the SAE. It is a common and stable optimization choice.

3 Experiments

3.1 MNIST

The dataset used throughout the entire project was the MNIST dataset. It is a standard benchmark dataset consisting of 70000 low resolution 28 x 28 images of handwritten digits. The dataset was loaded from PyTorch and transformed into a tensor of the shape (70000, 1, 28, 28) with all values ranging between [0, 1]. Loading it from PyTorch also includes all corresponding labels, which for my project will not be used for training and instead only for the evaluation of the clusters.

3.2 Clustering

For every experiment the decoder of the AE was discarded after training and only the encoder was used in the clustering. For VAEs the mean vector was used as the latent representation. k-means was always set with the number of clusters was set to 10, and n_init = 20 resulting in k-means running 20 times with different centroid seeds and choosing the best output. A random seed of 42 was used for reproducibility.

3.3 DEC - Autoencoder

The first autoencoder (AE) used in this project was a replication of the stacked autoencoder proposed in the paper. The paper uses the following SAE architecture: A stacked autoencoder with the dimensions $d \rightarrow 500 \rightarrow 500 \rightarrow 2000 \rightarrow 10 \rightarrow 2000 \rightarrow 500 \rightarrow 500 \rightarrow d$, with $d = 1 * 28 * 28$. Every dense layer is followed by the ReLU activation function, except for the decoder in the first pair $d \rightarrow 500 \rightarrow d$, and the encoder in the last layer $2000 \rightarrow 10 \rightarrow 2000$.

The SAE is pretrained with a greedy layer-wise approach, meaning only one AE layer at a time was trained with the deeper layers being disregarded in forward and backward propagation, and the weights of previous layers being frozen.

After pretraining all layers are unfrozen and updated during fine-tuning. It also only uses mean squared error (MSE) as its loss function for all the pretraining layers and fine-tuning.

3.3.1 Sigmoid. In the proposed architecture the last decoder $500 \rightarrow d$ does not have an activation function which caused artifacts in the reconstruction. Here I decided to use the sigmoid activation function in its final decoder output to bound the values in the same range as the input which is [0, 1]. This resulted in no visible artifacts in the final reconstructed images and increased clustering accuracy from 65% to 75%.

3.3.2 Binary Cross Entropy. Due to the input and now the output being in the range of [0, 1], I have decided to use a different loss

function for the decoder outputs that aim to reconstruct the original pixel values. Binary Cross Entropy (BCE) is commonly used for binary targets, but also works for soft targets in the range of [0, 1]. MSE was still kept for all the hidden layers.

In pretraining the first AE layer reconstructs the input and so does the SAE in fine-tuning, both of these stages now used BCE as their loss function.

This switch of loss function improved clustering accuracy from 75% to 79%.

3.4 Convolutional Layer Autoencoder

A different architecture I decided to explore was an autoencoder that utilizes convolutional layers instead of the fully connected layers used in the previous model. The reason is that convolutional layers are commonly used for images as they are effective at preserving local spatial structure, and could therefore help in preserving the structure of the digits when compressing.

I used two stacks of $Conv2d \rightarrow ReLU$ for my encoder and $ConvTranspose2d \rightarrow ReLU$ for the decoder. The final layer of the decoder uses sigmoid instead of ReLU for the same reasons as before.

The convolutional layers all consisted of kernel size 3, stride 2 and padding 1 to achieve a dimension reduction without max pooling. The features go from $[1, 28, 28] \rightarrow [32, 14, 14] \rightarrow [64, 7, 7]$ in the convolutional encoder to then being flattened to $(64 * 7 * 7)$ and compressed into the latent space of dimension 10 with a fully connected layer. The loss function is still BCE, but I have decided to use AdamW as the optimizer for all subsequent experiments due to its ease of use. The SGD and hyperparameters that were used in the SAE of the previous section were designed for that architecture, so instead of focusing on finding new hyperparameters for each experiment I chose AdamW.

This model gave a clustering accuracy of 80.36%, a very minor improvement over the SAE.

3.5 Variational Autoencoder

The final architecture that was explored are the variational autoencoders (VAEs), they were selected for their inherent focus on organizing the features in the latent space by regularizing it during training. This is achieved by the VAE loss function that has a reconstruction loss that ensures the decoder's output is similar to the input while also having a KL-divergence loss that pushes encoded distributions to adopt a normal distribution.

When encoding, a standard AE maps an input to a deterministic latent vector, the VAE instead outputs a mean and log-variance that defines a latent distribution from which a latent vector can be sampled.

Here the loss function is ELBO, the standard of VAE training, that consists of two components: the reconstruction loss function and the regularization term the KL-divergence. As in the previous implementations I used BCE for the reconstruction loss function and AdamW for the optimizer. For the clustering I used the mean vector as the deterministic latent representation.

To start off a simple model with the following structure $d \rightarrow 500 \rightarrow 500 \rightarrow 10$ was implemented. Followed by a deeper model that replicated the structure of the SAE. The less complex model yielded far superior results than the deeper one with a 78.68% clustering accuracy compared to a 64.24%.

3.5.1 Convolutional Layers. The same architecture that was implemented in section 3.3 was also implemented here for the VAE. To further improve the results an expansion was implemented with the β hyperparameter to determine the influence of the KL-divergence in the training. More specifically I tested this VAE with $\beta = 1$, $\beta = 0.5$, and a linear annealing version that increases β from 0 to 1 throughout the beginning of the training by setting it to be $min(1, (current\ epoch/50))$.

$\beta = 1$ achieved the best cluster accuracy of all models with an 81.79%, with the linear annealing model having a close accuracy of 81.28%, but better NMI and ARI results.

In an attempt to further improve the model, the complexity of $\beta = 1$ has been increased and an additional convolutional layer has been added $[64, 7, 7] \rightarrow [128, 4, 4]$ with the decoder mirroring that addition with ConvTranspose2d. This model gave the best results by far with a clustering accuracy of 85.73%.

4 Discussions

4.0.1 Feature Extraction. As a base for this project I took the AE + k-means model of the paper that achieved a 81.70%, which one of my models, namely the complex convolutional VAE, beat. Nevertheless several models managed to meaningfully map the dataset into a reduced latent space by achieving a clustering accuracy above 80%. The results of ARI and NMI metric support this observation. ARI measures pairwise agreement between predicted clusters and true labels, and NMI measures how much information the clusters contain about the true label.

The biggest increase in performance came from additions to the proposed DEC SAE. The sigmoid activation function in the output and the more specific BCE loss function for the training stages that compare original image with its reconstruction gave a 14% accuracy increase.

Utilizing convolutional layers instead of the DEC-style proposed fully connected AE structure achieved comparable results, and in future experiments the convolutional VAE's are the best performing. This shows that the intent behind using convolutional layers to better capture spatial structure when reducing to a latent space was an appropriate choice.

4.0.2 Similarity. MNIST has an ambiguity problem when it comes to certain digits. Pairs of digits such as 2, 3, 5 and 8, 4 and 9, and 7 and 9 often get grouped together in the latent space, which causes resulting clusters to have a high percentage of incorrectly assigned digits.

The same problems appeared in my experiments and this ambiguity is recognizable in many visualizations of the latent spaces using t-SNE. Many of these pairs are very close to each other as seen in Figure 2 with digits: 2 3, 5 and 8. Here we can see how cluster 3 assigns digit 8s into 3s, and the same can be observed for the digits 4, 7 and 9. For the model these digits are similar and cause the groupings of these representations in the latent to be less separable.

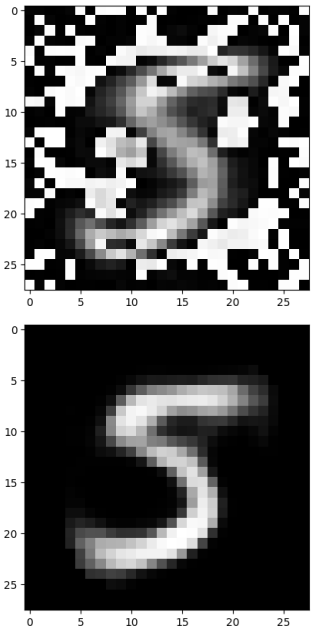


Figure 1: After fine-tuning reconstructions. Top without sigmoid, bottom with sigmoid.

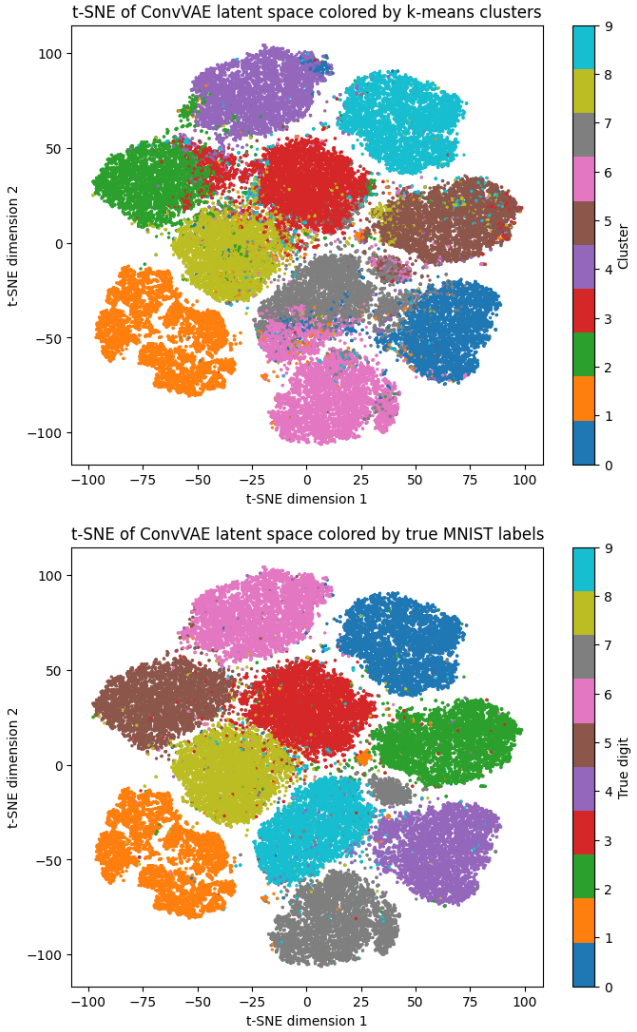


Figure 2: t-SNE visualization of latent space of convolutional VAE. Top predicted clusters, bottom true clusters.

References

[1] Irina Higgins, Loïc Matthey, Arka Pal, Christopher P. Burgess, Xavier Glorot, Matthew M. Botvinick, Shakir Mohamed, and Alexander Lerchner. 2017. beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework.. In *ICLR (Poster)*. OpenReview.net. http://dblp.uni-trier.de/db/conf/iclr/iclr2017.html#HigginsMPBGBML17

[2] G. E. Hinton and R. R. Salakhutdinov. 2006. Reducing the Dimensionality of Data with Neural Networks. *Science* 313, 5786 (July 2006), 504–507. doi:10.1126/science.1127647

[3] Lawrence Hubert and Phipps Arabie. 1985. Comparing Partitions. *Journal of Classification* 2, 1 (1985), 193–218. doi:10.1007/BF01908075

[4] Diederik P Kingma and Max Welling. 2022. Auto-Encoding Variational Bayes. arXiv:1312.6114 [stat.ML] https://arxiv.org/abs/1312.6114

[5] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324. doi:10.1109/5.726791

[6] Yann LeCun and Corinna Cortes. 2010. MNIST handwritten digit database. http://yann.lecun.com/exdb/mnist/. (2010). http://yann.lecun.com/exdb/mnist/

[7] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. arXiv:1711.05101 [cs.LG] https://arxiv.org/abs/1711.05101

[8] J. MacQueen. 1967. Some methods for classification and analysis of multivariate observations. https://api.semanticscholar.org/CorpusID:6278891

[9] Alexander Strehl and Joydeep Ghosh. 2002. Cluster Ensembles—A Knowledge Reuse Framework for Combining Multiple Partitions. *Journal of Machine Learning Research* 3 (2002), 583–617. https://www.jmlr.org/papers/v3/strehl02a.html

[10] Junyuan Xie, Ross B. Girshick, and Ali Farhadi. 2015. Unsupervised Deep Embedding for Clustering Analysis. *CoRR* abs/1511.06335 (2015). arXiv:1511.06335 http://arxiv.org/abs/1511.06335

A Hyperparameters

For all experiments the MNIST dataset was loaded into a dataloader of batch size 256. The epoch count for all experiments was 400, for section 3.2 the pretraining epochs for the greedy layer-wise training were 200. As given in the paper, SGD has a learning rate of 0.1 and momentum of 0.9. With the scheduler multiplying the lr every 20000 iterations by 0.1.

For the other models training used the AdamW optimizer with a learning rate of 0.001 and a weight decay of 0.0001.

For all experiments the dimension of the latent space was 10, so that it is comparable to the baseline of section 3.2

Table 1: Comparison of implemented autoencoder architectures on MNIST clustering.

Model	Main modification	Accuracy	ARI	NMI
SAE baseline	MSE reconstruction loss	65.00%	–	–
SAE + sigmoid output	Final decoder output bounded to $[0, 1]$	75.00%	60.10%	66.87%
SAE + sigmoid + BCE	BCE for image reconstruction	79.00%	66.42%	72.56%
Convolutional AE	Two convolutional encoder layers	80.36%	67.98%	73.17%
Simple VAE	Fully connected VAE, $d \to 500 \to 500 \to 10$	78.68%	64.31%	68.44%
Deeper VAE	VAE using SAE-like dense architecture	64.24%	47.87%	53.83%
ConvVAE, $\beta = 1$	Convolutional VAE with fixed KL weight	81.79%	66.67%	70.96%
ConvVAE, linear annealing	β increased from 0 to 1 during training	81.28%	67.59%	72.41%
Expanded ConvVAE, $\beta = 1$	Added third convolutional layer	**85.73%**	**72.92%**	**75.53%**